

SEIP_ASSIGNMENT_1_2026

1. GitHub Repository Link:

https://github.com/poullakkos/seip_assignment_1_2026

2. CI/CD Proof:

The top screenshot shows the GitHub Actions CI/CD workflow interface for a workflow named 'CI/CD workflow done #1'. The workflow is in a 'Success' state, triggered by a push to the 'main' branch. The total duration of the workflow is 24s. The 'build' job is highlighted in the left sidebar, and its details are shown in the main panel. The 'ci-cd.yaml' file is listed with a 'build' step that took 20s.

The bottom screenshot shows the 'Annotations' tab for the same workflow. It displays a list of steps and their durations:

Step	Duration
Set up job	1s
Checkout code	0s
Login to GitHub Container Registry (GHCR)	1s
Build and push	15s
Post Build and push	0s
Post Login to GitHub Container Registry (GHCR)	0s
Post Checkout code	0s
Complete job	1s

3. Cluster State Proof:

```
C:\Users\Vasilis Poullakkos\seip_assignment_1_2026>kubectl get all -n default
```

NAME	READY	STATUS	RESTARTS	AGE
pod/deployment-7565b9f8d5-6n87p	1/1	Running	2 (4d2h ago)	4d3h
pod/deployment-7565b9f8d5-g4ckd	1/1	Running	2 (4d2h ago)	4d3h
pod/deployment-7565b9f8d5-zh2z4	1/1	Running	2 (4d2h ago)	4d3h

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4d3h
service/service	ClusterIP	10.105.215.54	<none>	80/TCP	4d3h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/deployment	3/3	3	3	4d3h

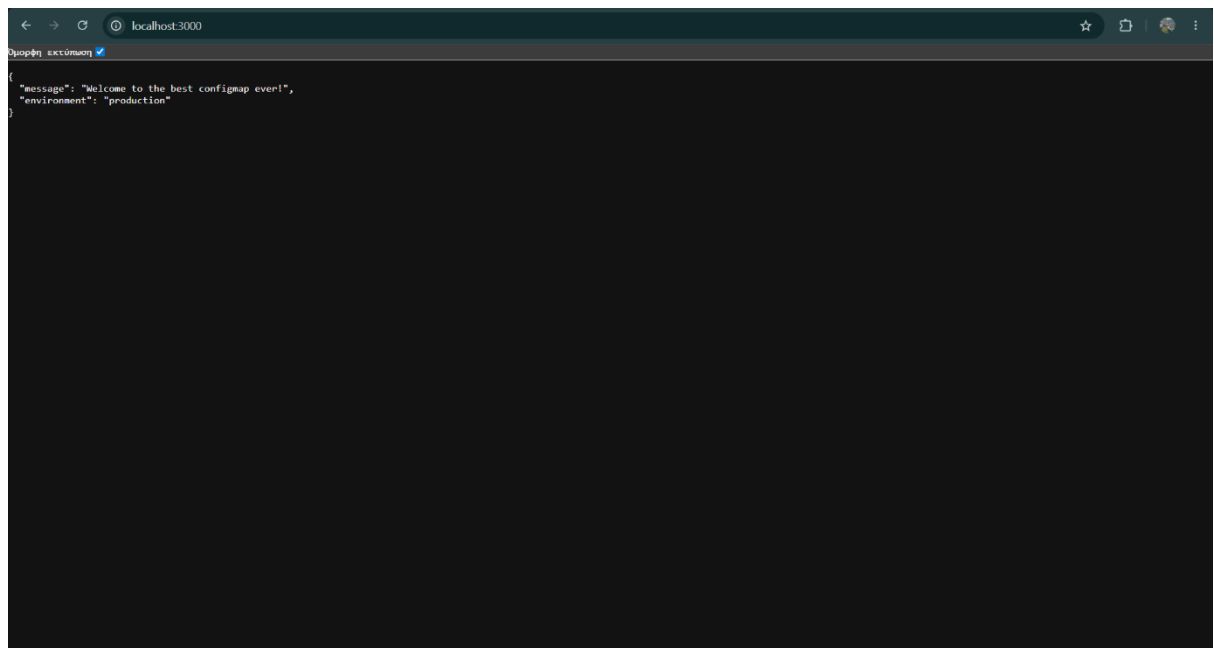
NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/deployment-7565b9f8d5	3	3	3	4d3h

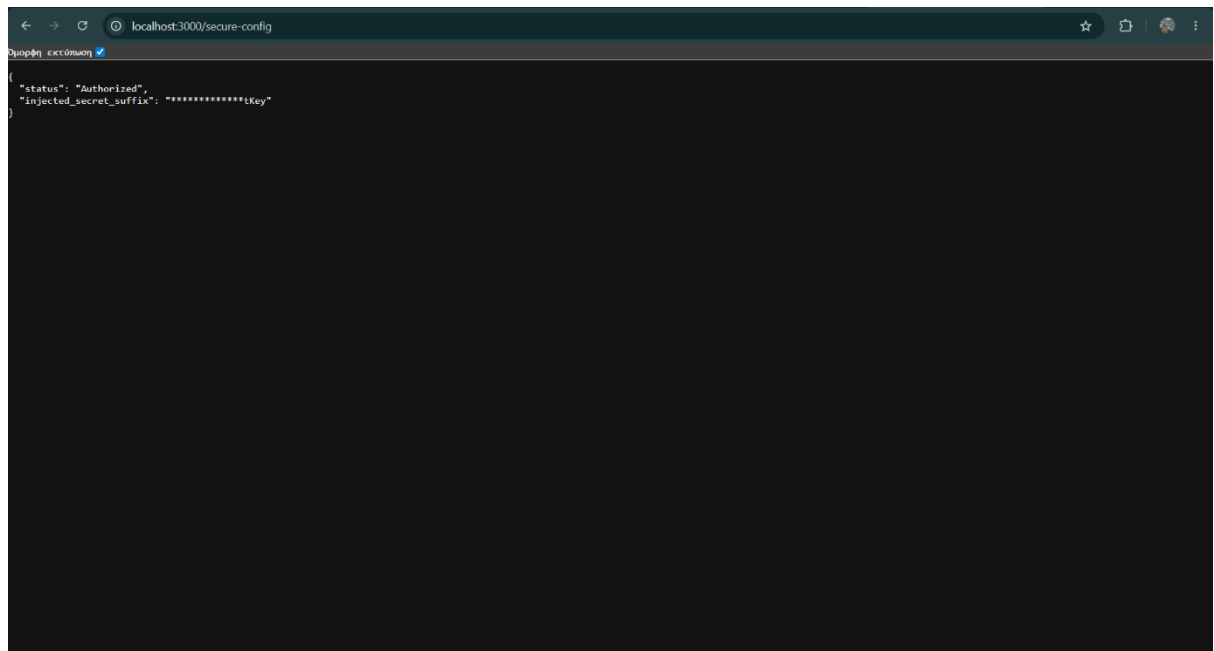

```
C:\Users\Vasilis Poullakkos\seip_assignment_1_2026>kubectl get configmap,secret
```

NAME	DATA	AGE
configmap/configmap	2	4d3h
configmap/kube-root-ca.crt	1	4d3h

NAME	TYPE	DATA	AGE
secret/secret	Opaque	1	4d3h

4. Application Verification Proof:





5. AI Reflection & Future Outlook:

- **AI Integration:** For this assignment, I primarily used AI to guide me and explain what I was actually building at each step. Subsequently, I used it to review my code, write the documentation for this report, and finally to clarify terminology and concepts I could not fully understand on my own. I did not use it for code generation, but strictly as a supporting tool that guided me throughout the assignment.
- **Utility Analysis:** The aspects of AI assistance that proved most valuable were syntax debugging, as I encountered several syntactical errors in my code that would have prevented it from functioning correctly, such as missing spaces or incorrect indentation (tabs) required in each file. It also helped me considerably in understanding Docker and Kubernetes concepts and how to use them in practice. Finally, it assisted me with the exact terminal commands I needed to type in CMD to verify that everything was working correctly.
- **Friction Points:** The AI was generally helpful throughout the assignment, however there were a few points where I had to troubleshoot manually. First, the GitHub Actions workflow initially failed with a denied: installation not allowed to Create organization package error, which the AI did not anticipate. I had to manually navigate to the repository Settings and enable Read and Write permissions for workflow actions. Second, after deleting the Minikube container directly through Docker Desktop instead of using minikube stop, which the AI had assured me would be fine, the cluster entered a broken state with connection refused errors. The AI suggested running minikube delete followed by minikube start to fully reset the environment.

- **Future Architectural Outlook:** If I had an extra week or was tasked with scaling this pipeline for a real-world enterprise system, the first thing I would focus on is making the deployment truly zero-downtime by implementing proper rolling update strategies with health checks that prevent a bad release from ever reaching users. I would also move away from storing secrets inside Kubernetes manifests entirely and instead integrate a dedicated secrets management solution, since even Base64 encoded secrets inside a repository are not truly secure in a professional environment. Another step I would take is setting up separate environments for staging and production, so that every change gets tested in a staging cluster before it ever touches production. Finally, I would add automated testing as a mandatory step inside the CI pipeline, meaning that if the tests fail, the image never gets built or pushed at all, which would make the entire pipeline much more reliable and trustworthy.